Un modèle qui dessine directement en espace écran

Proposition d'orientation et plan d'action

Melissa Colin
projet « Graph drawing différentiable », ENSEIRB-MATMECA 3A

9 octobre 2026

Document de travail, à discuter avec l'équipe et M. Giot.

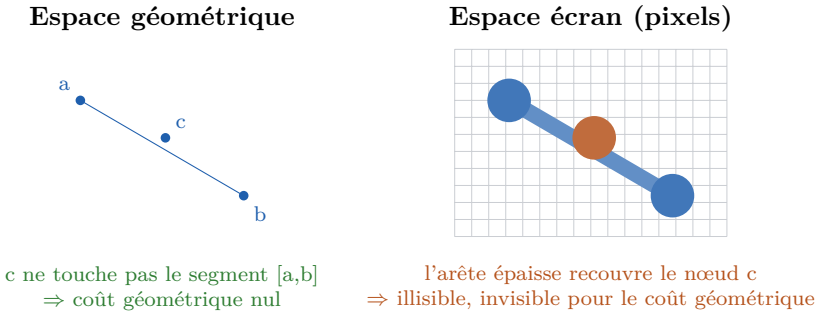
En bref. Je propose d'entraîner un réseau de neurones sur graphes qui prédit directement la position de chaque nœud dans l'image finale, en coordonnées écran, au lieu d'optimiser chaque graphe séparément. Le réseau apprend avec des fonctions de coût calculées sur l'image rendue (rendu différentiable par SDF), donc sur ce que l'œil voit.

En fin de projet, s'il reste du temps, on l'affine selon nos préférences visuelles, un critère qu'on ne sait pas écrire en formule, à travers un modèle critique appris.

Côté organisation, ça permet de travailler en parallèle : une personne sur le modèle, une sur le rendu et les coûts, une sur l'évaluation, avec une interface commune fixée dès le départ.

1 Le problème, en une figure

Les méthodes classiques ((GD)^2, SGD, FORBID) optimisent un dessin dans un espace géométrique continu : un nœud est un point, une arête est un segment d'épaisseur nulle. Mais le dessin finit affiché en pixels, avec des nœuds de taille fixe, des arêtes épaisses et une résolution limitée. Un dessin « bon » en géométrie peut être illisible à l'écran.

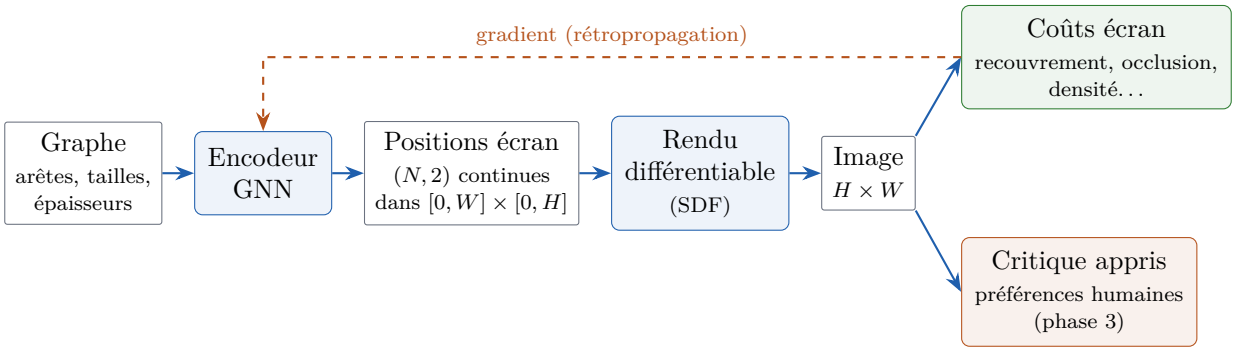


Le sujet propose de calculer les coûts sur l'image rendue. Pour que le gradient remonte des pixels jusqu'aux positions, le rendu doit être différentiable : chaque nœud et chaque arête est décrit par une fonction de distance signée (SDF) $d(p)$, transformée en intensité douce $I(p) = \sigma(-d(p)/\varepsilon)$.

2 L'idée proposée

2.1 Un modèle qui pense en pixels

Optimiser les positions de *chaque* graphe par descente de gradient coûte cher, et il faut tout recommencer pour le graphe suivant. Je propose plutôt d'entraîner une fois pour toutes un réseau qui produit le dessin d'un coup :



Le réseau reçoit la structure du graphe et ses attributs (taille des nœuds, épaisseur des arêtes). Un GNN accepte un nombre de nœuds variable, ce qui règle la question des graphes de tailles différentes. En sortie, il donne des coordonnées continues en pixels. On commence avec une taille d'image fixe, puis on cherchera comment passer la résolution en entrée du modèle.

Ce qui m'intéresse le plus : que l'architecture garantisse certaines contraintes, comme l'absence de chevauchement, au lieu de seulement les pénaliser dans la fonction de coût.

2.2 Trois phases d'entraînement



Aucune phase n'a besoin de dessins de référence. Le modèle produit un dessin, la loss le note, le gradient corrige les poids : DeepGD entraîne déjà un GNN de cette façon, directement sur des critères esthétiques. La phase 1 sert à démarrer. Les coûts en pixels construits avec des SDF ont un gradient presque nul quand les éléments sont loin les uns des autres (la sigmoïde est plate loin du bord), donc un réseau qui place ses nœuds au hasard recevrait très peu de signal. Le stress, lui, donne un gradient partout et apprend d'abord une structure globale correcte.

Phase 3 en détail. On annote des paires de dessins (« je préfère A à B »), on entraîne un modèle critique (par exemple un CNN sur l'image) à prédire ces préférences, puis on affine le modèle de dessin pour maximiser ce score, en le gardant proche du modèle de la phase 2 (pénalité de divergence) pour qu'il ne « triche » pas. On peut s'y prendre de trois façons :

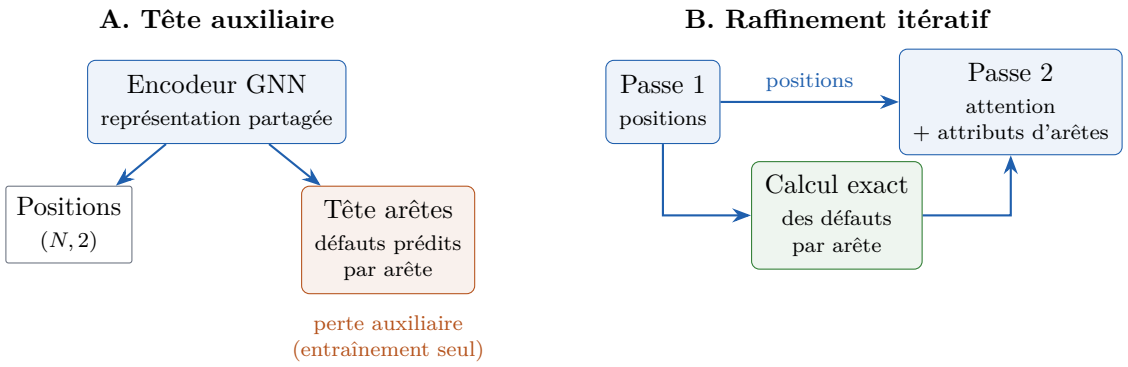
— par RL (PPO ou REINFORCE), comme le RLHF des modèles de langage : le modèle sort une distribution sur les positions, on tire des dessins, le critique les note ;

— par DPO, qui apprend directement à partir des paires de préférences, sans critique ni boucle de RL et reste plus stable avec peu de données ;

— par rétropropagation directe : si le critique et le rendu sont différentiables, on peut rétropropager son score jusqu'au modèle (approche de type DRaFT ; sur leur tâche de référence, les auteurs mesurent un gain de vitesse de plus de 200 fois par rapport au RL).

2.3 Rendre le modèle conscient des arêtes

Les arêtes ne sont pas prédites : ce sont des segments droits entre les nœuds. Pourtant, la plupart des défauts propres à l'écran viennent d'elles (arête qui cache un nœud, traits qui fusionnent, croisements). Je voudrais que le modèle raisonne explicitement sur les arêtes pendant qu'il place les nœuds. Je vois deux façons de le faire :



— **A. Tête auxiliaire (multi-tâche).** Une tête supplémentaire prédit, pour chaque arête, ses défauts dans le dessin final : nombre de croisements, occlusion d'un nœud, longueur à l'écran. Elle est entraînée par une perte auxiliaire (cible calculée exactement sur le dessin produit) et ne sert qu'à l'entraînement. Elle force la représentation partagée à « comprendre » les arêtes, et ses prédictions peuvent aussi être réinjectées dans l'attention.

— **B. Raffinement itératif.** Le modèle propose des positions, on calcule exactement les défauts de chaque arête (peu coûteux), puis on les fournit comme attributs d'arêtes à une seconde passe qui corrige. Pas d'approximation apprise, mais deux passes ou plus.

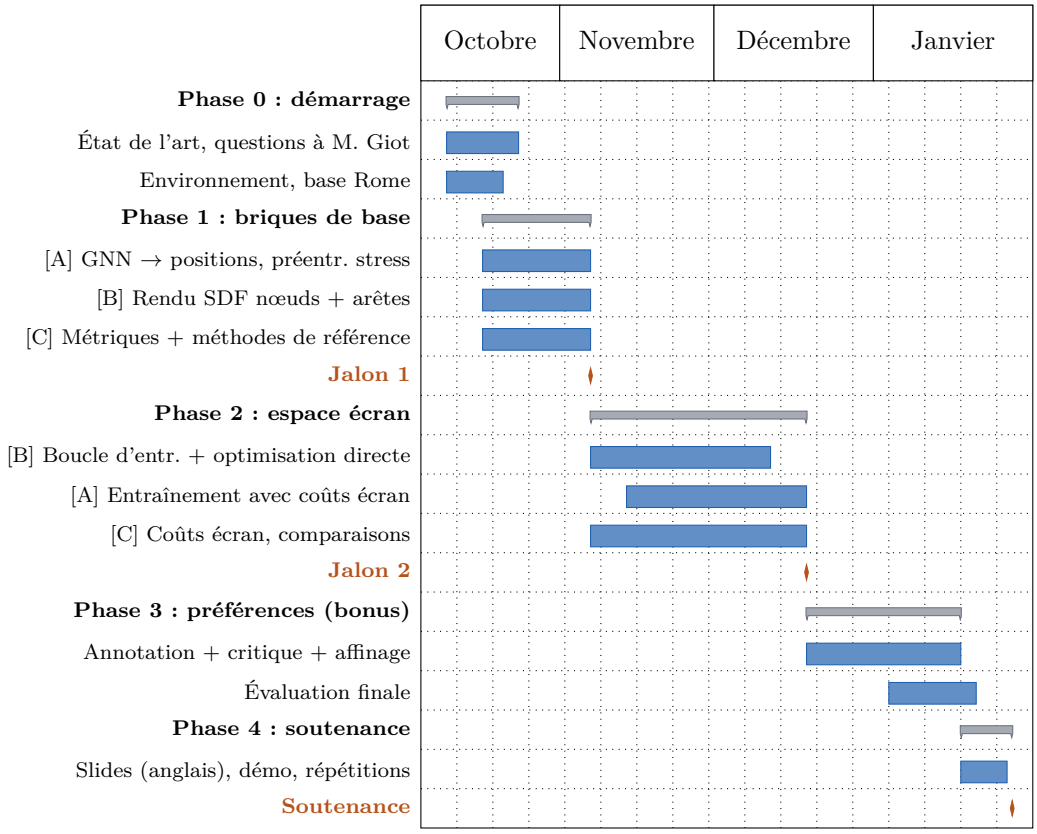
Hypothèse à tester. Le coût calculé sur l'image pénalise déjà les défauts d'arêtes. Reste à savoir si rendre cette information *explicite* accélère l'apprentissage ou améliore le résultat. Protocole : comparer le modèle de base, la variante A et la variante B sur les mêmes graphes et les mêmes métriques (*ablation*).

3 Points de vigilance et parades

Risque	Parade envisagée
Pixels distincts $\neq$ pas de chevauchement : un nœud est un disque de r pixels ; deux nœuds sur des pixels voisins se recouvrent quand même.	La contrainte réelle est $\|c_i - c_j\| \geq r_i + r_j$. À traiter par un coût de recouvrement sur l'image, ou par une couche de projection / répulsion en sortie du réseau.
Une sortie discrète bloque le gradient : choisir un pixel revient à un arg max, de gradient nul.	Sortir des coordonnées continues (ou des cartes de probabilité douces, Gumbel-softmax). Voir une affectation « un nœud par case » comme un problème d'affectation différentiable (Sinkhorn).
Les arêtes ne sont pas prédites : ce sont des segments droits entre nœuds ; occlusions et fusions de traits viennent d'elles.	Le modèle les contrôle indirectement, via les nœuds ; les coûts sur l'image (occlusion arête-nœud, densité) font remonter le gradient vers les extrémités. En plus : tête auxiliaire ou raffinement itératif (section 2.3).
Mémoire et calcul : de l'ordre de $(N+E) \times H \times W$ par graphe.	Petits graphes (base Rome), résolution modeste au début ; calcul des recouvrements via la carte somme $S = \sum_i I_i$ plutôt que paire par paire.
Préférences humaines : il faut beaucoup d'annotations, et trois annotateurs ont des goûts biaisés.	Phase 3 traitée comme un bonus ; DPO ou critique préentraîné sur des métriques connues (lisibilité, cf. Haleem et al.) pour réduire le besoin d'annotations.
Évaluer un modèle appris est plus long qu'une optimisation directe.	Une personne chargée de la loss et de l'évaluation dès le début, avec des métriques indépendantes de la loss (section 4.3).

4 Plan d'action

4.1 Calendrier



4.2 Répartition des tâches

Rôle	Responsabilité
A. Pipeline : modèle (Melissa)	Architecture GNN (graphe → positions écran), préentraînement par le stress, entraînement avec les coûts écran, phase 3.
B. Pipeline : rendu	Rendu différentiable (SDF ou autre méthode si plus adaptée), boucle d'entraînement, optimisation directe d'un graphe, base Rome.
C. Loss et évaluation	Coûts écran, métriques, méthodes de référence classiques, comparaisons, mesure d'impact environnemental.

Interface commune. À figer dès la phase 0, pour que chacun avance sans attendre les autres :

- le modèle produit `positions` : tenseur $(N, 2)$, valeurs continues dans $[0, W] \times [0, H]$;
- `render(positions, graphe, attributs)` → `image` (C, H, W), différentiable ;
- chaque coût : `loss(image, ...)` → `scalaire`.

En attendant le rendu (B), le rôle A s'entraîne avec le stress seul, ce qui correspond de toute façon à la phase 1.

4.3 Règle d'évaluation

La loss et l'évaluation sont confiées à la même personne parce qu'elles reposent sur les mêmes mesures. Une règle s'impose alors : un modèle évalué avec les termes qu'il minimise gagne forcément, puisqu'il a été entraîné pour ça (loi de Goodhart). L'évaluation inclut donc :

— des métriques absentes de la loss d'entraînement (par exemple les croisements, si on ne les optimise pas, et des métriques de lisibilité de la littérature) ;

— une comparaison aux méthodes de référence (SGD, FORBID…) sur les mêmes graphes, mesurée sur le rendu final.

5 Questions pour M. Giot

Orientation

1. Nous voulons entraîner un modèle (GNN) qui prédit directement les positions en espace écran, sans supervision, avec des coûts calculés sur l'image rendue. Cette orientation vous convient-elle, ou préférez-vous que nous commencions par l'optimisation directe d'un graphe ?
2. Quelles entrées viser au départ : seulement la structure (nœuds, arêtes) et la taille des nœuds, ou aussi les épaisseurs, les couleurs et les formes ?
3. Quelle taille de graphes viser (nombre de nœuds), quelle résolution d'image, et quel temps de calcul vous paraîtrait acceptable ?

Évaluation

4. Quelles métriques et quel protocole recommandez-vous pour comparer un dessin optimisé en espace écran à un dessin géométrique ?
5. Quelles méthodes de référence comparer en priorité, et existe-t-il des implémentations que nous pouvons réutiliser ?

Organisation et rendus

6. Qu'attendez-vous aux jalons du 06/11 et du 18/12 ? Un état de l'art écrit est-il attendu, ou seulement une présentation orale ?
7. Quels livrables finaux attendez-vous : le code seul, un notebook de démonstration, un rapport ?
8. Le sujet mentionne des exemples de code : comment y accéder ?
9. Disposons-nous de GPU (LaBRI ou école), ou travaillons-nous sur nos machines ?
10. Le sujet mentionne GitLab : devons-nous l'utiliser, ou un dépôt GitHub privé auquel nous vous donnons accès vous convient-il ?

6 Ressources pour creuser

Modèles appris pour le dessin de graphes

— X. Wang, K. Yen, Y. Hu, H.-W. Shen. *DeepGD : A Deep Learning Framework for Graph Drawing Using GNN.* IEEE CG&A, 2021. Un GNN entraîné directement sur des critères esthétiques.

— X. Wang et al. *SmartGD : A GAN-Based Graph Drawing Framework for Diverse Aesthetic Goals.* IEEE TVCG, 2023. Un discriminateur apprend ce qu'est un « bon » dessin.

— L. Giovannangeli et al. *Toward Efficient Deep Learning for Graph Drawing (DL4GD).* IEEE TVCG. Travail de l'équipe de M. Giot.

Juger un dessin à partir de son image (base du critique)

— H. Haleem, Y. Wang, A. Puri, S. Wadhwa, H. Qu. *Evaluating the Readability of Force Directed Graph Layouts : A Deep Learning Approach.* IEEE CG&A 39(4), 2019. Un CNN prédit des métriques de lisibilité à partir de l'image du graphe.

Rendu différentiable

— I. Quilez. *2D distance and gradient functions*, 2019. `https://iquilezles.org/articles/distgradfunctions2d/`

— T.-M. Li, M. Lukáč, M. Gharbi, J. Ragan-Kelley. *Differentiable Vector Graphics Rasterization for Editing and Learning (DiffVG).* ACM TOG, 2020. Rasteriseur différentiable de formes vectorielles, une alternative possible aux SDF.

Optimisation géométrique (méthodes de référence)

— J. X. Zheng, S. Pawar, D. F. M. Goodman. *Graph Drawing by Stochastic Gradient Descent.* IEEE TVCG, 2018.

— R. Ahmed et al. *Graph Drawing via Gradient Descent, $(GD)^2$.* Graph Drawing, 2020.

— L. Giovannangeli et al. *FORBID : Fast Overlap Removal By Stochastic Gradient Descent for Graph Drawing.* Graph Drawing, 2022.

Affinage par préférences humaines

— P. Christiano et al. *Deep Reinforcement Learning from Human Preferences.* NeurIPS, 2017. Apprend une récompense à partir de comparaisons par paires.

— L. Ouyang et al. *Training Language Models to Follow Instructions with Human Feedback (InstructGPT).* NeurIPS, 2022. L'enchaînement supervisé → critique → RL.

— R. Rafailov et al. *Direct Preference Optimization.* NeurIPS, 2023. Préférences sans critique ni RL.

— K. Clark, P. Vicol, K. Swersky, D. J. Fleet. *Directly Fine-Tuning Diffusion Models on Differentiable Rewards (DRaFT).* ICLR, 2024. Rétropropager une récompense différentiable plutôt que faire du RL.

Architecture : arêtes et tâches auxiliaires

— V. P. Dwivedi, X. Bresson. *A Generalization of Transformer Networks to Graphs.* AAAI Workshop on Deep Learning on Graphs, 2021. Attention sur graphes avec attributs d'arêtes (variante B).

— M. Jaderberg et al. *Reinforcement Learning with Unsupervised Auxiliary Tasks.* ICLR, 2017. Tâches auxiliaires pour enrichir la représentation partagée (variante A).

Sorties discrètes différentiables

— E. Jang, S. Gu, B. Poole. *Categorical Reparameterization with Gumbel-Softmax.* ICLR, 2017.

— G. Mena, D. Belanger, S. Linderman, J. Snoek. *Learning Latent Permutations with Gumbel-Sinkhorn Networks.* ICLR, 2018.